# EntraID / 3LO: AC Gateway + AC Identity

## Volume 2 — Reference Implementation, Credential Vault & Operational Runbook

GENAIAF-562 · GENAI Agent Foundry · June 2026

| | |
|---|---|
| **Continues** | Volume 1 — Standards, Architecture, Industry Analysis, Dependency Tracker |
| **This volume** | Sections 11–17: Code, Sidecar Pattern, Credential Vault, Ops Runbook, Incident Response, OWASP Agentic Top |
| **Audience** | Engineering leads, Platform/SRE, Security |

---

## 11. REFERENCE IMPLEMENTATION — CODE PATTERNS

### 11.1 The Sidecar Auth Pattern (Language-Agnostic)

The Microsoft Entra SDK for Agent ID runs as a **containerised sidecar** alongside the agent. The agent never handles tokens directly — it makes a simple HTTP call to the sidecar, which performs the full FIC→OBO chain and returns an Authorization header. This pattern works for Python, Node.js, Go, Java, and any other language without embedding identity SDK logic into application code.

| Sidecar endpoint | Purpose | Query params |
|---|---|---|
| /AuthorizationHeader/Graph | Get delegated token for MS Graph (OBO) | AgentIdentity={guid}, AgentUserId={oid} or AgentUsername={upn} |
| /AuthorizationHeader/Graph (no params) | Standard OBO — use incoming user token identity | None (inherits from Bearer token) |
| /AuthorizationHeader/Graph?AgentIdentity=… | Autonomous agent-only token (no user context) | AgentIdentity={guid} only |
| /Validate | Validate inbound user/agent token | None — pass token in Authorization header |
| /DownstreamApi | Sidecar directly calls downstream API and returns result | url, AgentIdentity, AgentUserId |

### 11.2 .NET — Microsoft.Identity.Web + Agent Identities

Using the official SDK. This handles the two-step FIC token exchange internally:

```
// Program.cs — service registration
builder.Services.AddMicrosoftIdentityWebApiAuthentication(builder.Configuration)
.EnableTokenAcquisitionToCallDownstreamApi() .AddAgentIdentities() // <-- enables agent OBO patterns
.AddInMemoryTokenCaches(); // Controller / endpoint — OBO for user via Agent Identity
app.MapGet("/jira/issue/{key}", async (HttpContext ctx, string key) => { string agentIdentityId =
""; var authProvider = ctx.RequestServices .GetRequiredService(); // Options bind agent identity to
this OBO call var options = new AuthorizationHeaderProviderOptions()
.WithAgentIdentity(agentIdentityId); // SDK performs: Blueprint acquires T1, then Agent Identity OBO
exchange string authHeader = await authProvider .CreateAuthorizationHeaderForUserAsync(
["https://graph.microsoft.com/.default"], options); // Use token to call downstream — in this case
Graph, swap for Jira token via vault httpClient.DefaultRequestHeaders.Authorization =
AuthenticationHeaderValue.Parse(authHeader); return await httpClient.GetAsync(
$"https://api.atlassian.com/ex/jira/{{cloudId}}/rest/api/3/issue/{{key}}"); })
.RequireAuthorization(); // NuGet packages required: // dotnet add package Microsoft.Identity.Web //
dotnet add package Microsoft.Identity.Web.AgentIdentities // dotnet add package
Microsoft.Identity.Web.GraphServiceClient
```

✓ WithAgentIdentity(guid) switches the OBO exchange from Blueprint-level to Agent Identity-level, giving each instance a distinct sub claim and audit trail.

## 11.3 Python — MSAL OBO Flow (Direct)

For teams not using the sidecar, MSAL Python implements OBO directly:

```
import msal, requests # --- Step 1: Blueprint acquires T1 using MSI / FIC --- # (In Azure, use
DefaultAzureCredential; below shows client_credentials fallback for dev) blueprint_app =
msal.ConfidentialClientApplication( client_id=BLUEPRINT_CLIENT_ID,
authority=f"https://login.microsoftonline.com/{TENANT_ID}", client_credential={"thumbprint":
CERT_THUMBPRINT, "private_key": PRIVATE_KEY}, ) t1_result = blueprint_app.acquire_token_for_client(
scopes=["api://AzureADTokenExchange/.default"] ) T1 = t1_result["access_token"] # --- Step 2: Agent
Identity OBO exchange (T1 + Tc → resource token) --- # Tc = user access token received from the
calling client app obo_payload = { "grant_type": "urn:ietf:params:oauth:grant-type:jwt-bearer",
"client_id": AGENT_IDENTITY_CLIENT_ID, "client_assertion_type":
"urn:ietf:params:oauth:client-assertion-type:jwt-bearer", "client_assertion": T1, # Agent identity
proves itself with T1 "assertion": user_access_token_Tc, # User's token to exchange
"requested_token_use": "on_behalf_of", "scope": "https://graph.microsoft.com/Mail.Read
offline_access", } resp = requests.post(
f"https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/token", data=obo_payload )
resource_token = resp.json()["access_token"] refresh_token = resp.json().get("refresh_token") #
Store in credential vault # --- Step 3: Call resource (Graph example) --- graph_resp = requests.get(
"https://graph.microsoft.com/v1.0/me/messages", headers={"Authorization": f"Bearer
{resource_token}"} ) # IMPORTANT: Never log access_token or refresh_token values. # Store
refresh_token in encrypted vault keyed by (user_oid, provider).
```

■ Never use Azure CLI tokens (Directory.AccessAsUser.All) for Agent Identity APIs — causes hard 403. Always use fmi_path with client_credentials, NOT RFC 8693 token-exchange grant (returns AADSTS82001). Always use /.default scope in both exchange steps — individual scope strings fail.

## 11.4 Python — Sidecar HTTP Pattern (Polyglot-safe)

```
import httpx SIDECAR_URL = "http://localhost:7000" # Never expose sidecar externally async def
get_delegated_graph_token(user_token: str, agent_identity_id: str) -> str: """Ask the Entra SDK
sidecar for a delegated Graph auth header.""" resp = await httpx.AsyncClient().get(
f"{SIDECAR_URL}/AuthorizationHeader/Graph", headers={"Authorization": f"Bearer {user_token}"},
params={"AgentIdentity": agent_identity_id} ) resp.raise_for_status() return resp.text # Returns
'Bearer eyJ...' ready to use async def call_graph_api(user_token: str, agent_id: str, path: str):
"""Generic Graph call via sidecar — agent never touches raw token.""" auth_header = await
get_delegated_graph_token(user_token, agent_id) return await httpx.AsyncClient().get(
f"https://graph.microsoft.com/v1.0{path}", headers={"Authorization": auth_header} ) # Atlassian
example — token comes from credential vault, NOT sidecar async def call_jira_api(user_oid: str,
cloud_id: str, issue_key: str, vault): """Retrieve Atlassian token from vault; refresh if needed."""
cred = await vault.get(user_oid, provider="atlassian") access_token = await vault.ensure_fresh(cred)
# handles proactive refresh return await httpx.AsyncClient().get(
f"https://api.atlassian.com/ex/jira/{cloud_id}/rest/api/3/issue/{issue_key}",
```

```
headers={"Authorization": f"Bearer {access_token}"} )
```

- The sidecar must run on localhost with network access restricted. Never expose via LoadBalancer, Ingress, or public endpoint.

## 12. CREDENTIAL VAULT — DESIGN & SCHEMA

### 12.1 Why Tokens Must Never Live in the Agent

In April 2026, an AI coding agent deleted a Railway production database after finding a long-lived API token in an unrelated file. The token had account-scoped permissions with no environment isolation. The deletion took nine seconds. This is the canonical example of why **credential separation is not optional**.

| Anti-pattern | Risk | Correct pattern |
|---|---|---|
| Token in environment variable | Exposed to all processes in container; leaked in logs | Encrypted vault; agent requests token via API call |
| Token in LLM context / prompt | LLM may echo token; prompt injection extracts it | Agent calls vault SDK; token injected into HTTP client, never into prompt |
| Shared service account token | One compromise = full org access; no per-user audit trail | Per-user, per-provider token stored with (user_oid, provider) key |
| Long-lived static token (GHE OAuth App) | Compromise window = token lifetime (days/forever) | Short-TTL tokens; GitHub App installation token (1hr); mandatory rotation |
| Token stored in agent memory across tasks | Agent retains access after task completes; overprivilege accumulates | Revoke / expire refresh token on task completion |

### 12.2 Vault Schema Design

```
-- Credential vault table (PostgreSQL / Azure SQL) CREATE TABLE agent_credentials ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_oid VARCHAR(64) NOT NULL, -- Entra user Object ID provider VARCHAR(32) NOT NULL, -- 'atlassian' | 'github' | 'graph' | 'ghe' cloud_id VARCHAR(64), -- Atlassian site cloudId (nullable) access_token BYTEA NOT NULL, -- AES-256-GCM encrypted refresh_token BYTEA, -- AES-256-GCM encrypted; nullable for GHE OAuth scopes TEXT[] NOT NULL, -- e.g. ARRAY['read:jira-work','offline_access'] expires_at TIMESTAMPTZ NOT NULL, -- absolute expiry of access_token refresh_by TIMESTAMPTZ NOT NULL, -- expires_at - 20% of TTL (proactive refresh point) token_version BIGINT NOT NULL DEFAULT 1, -- for optimistic locking on rotation agent_id VARCHAR(64) NOT NULL, -- which agent identity requested this consent_granted TIMESTAMPTZ NOT NULL, -- when user first granted consent last_used TIMESTAMPTZ, -- for access review / dormancy detection revoked_at TIMESTAMPTZ, -- set on user revoke or task completion created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), UNIQUE (user_oid, provider, cloud_id) -- one active credential per user+provider+site ); CREATE INDEX idx_cred_refresh ON agent_credentials(refresh_by) WHERE revoked_at IS NULL; -- background sweeper uses this index
```

### 12.3 Token Refresh Orchestration — Proactive at 80% TTL

Waiting for a 401 to trigger refresh causes cascading retries, race conditions, and agent task failures. The correct pattern is **proactive refresh before expiry** with **per-(user, provider) distributed locking** to prevent multiple concurrent refreshes.

```
import asyncio, time from redis.asyncio import Redis redis = Redis() # or Azure Cache for Redis in production async def ensure_fresh_token(vault, user_oid: str, provider: str) -> str: """Return a valid access_token, refreshing proactively if within 20% of TTL.""" cred = await vault.get(user_oid, provider) if cred.expires_at > time.time() + 60: return decrypt(cred.access_token) # Still valid # Need refresh — acquire per-(user, provider) lock to prevent races lock_key = f"token_refresh_lock:{user_oid}:{provider}" async with redis.lock(lock_key, timeout=15, blocking_timeout=10): # Re-read after acquiring lock — another worker may have already refreshed cred = await vault.get(user_oid, provider) if cred.expires_at > time.time() + 60: return decrypt(cred.access_token) # Refreshed by another worker # Actually refresh new_access, new_refresh, new_expiry = await refresh_from_provider( provider, decrypt(cred.refresh_token) ) # ATOMIC rotation — write new tokens, increment version, fail if version mismatch await vault.rotate( user_oid=user_oid, provider=provider, expected_version=cred.token_version, new_access=encrypt(new_access), new_refresh=encrypt(new_refresh), new_expiry=new_expiry,
```

```
new_refresh_by=new_expiry - (new_expiry - time.time()) * 0.20 ) return new_access # Background sweeper — runs every 5 minutes async def refresh_sweeper(vault): while True: due = await vault.list_due_for_refresh() # WHERE refresh_by < NOW() + 5min for cred in due: asyncio.create_task(ensure_fresh_token(vault, cred.user_oid, cred.provider)) await asyncio.sleep(300)
```

## 12.4 Encryption & Key Management

| Layer | Mechanism | Notes |
|---|---|---|
| Encryption at rest | AES-256-GCM per token field | Key stored in Azure Key Vault / AWS KMS; never in application config |
| Key hierarchy | Data Encryption Key (DEK) wrapped by Key Encryption Key (KEK) | KEK rotated annually; DEK rotated on each token write |
| Key access | Managed Identity granted Key Vault 'get' on DEK only | Agent service never has Key Vault admin rights |
| Audit | Every decrypt operation logged to Key Vault audit log | Correlate with agent_credentials.last_used for anomaly detection |
| Transit encryption | TLS 1.3 on all vault API calls | Internal cluster traffic also encrypted (mTLS) |
| No plaintext logging | Structured logs must mask token fields | Use log sanitiser middleware; assert in CI that token strings don't appear in test logs |

## 13. CONSENT FLOW UX — DESIGN PATTERNS

### 13.1 Three Consent Models

| Model | When to use | Trigger | User experience |
|---|---|---|---|
| Admin pre-consent | Agent accesses org-level data; no user-specific data; internal analytics | Admin grants consent once in Entra portal for all users | Invisible to end user. Requires admin to explicitly approve agent scopes. |
| User just-in-time consent (3LO) | Agent accesses user's personal resources (email, calendar, Jira tickets assigned to them) | First time agent attempts to use that resource for that user | User sees consent screen: 'Agent X wants to access your Jira on Bank. Allow?' — must explicitly approve. |
| CIBA — backchannel approval | High-risk or irreversible action (delete, send bulk, financial) | Before executing the risky tool call | Push notification to user's phone: 'Agent wants to send this email to 500 people. Approve / Deny.' |

### 13.2 Consent Screen Requirements (GDPR + Zero Trust)

| Requirement | Detail |
|---|---|
| Name the agent | Consent screen MUST show the specific agent name (not just the app). IETF draft requested_actor parameter enables this. |
| Scope plain language | Don't show 'write:jira-work'. Show: 'Create and update your Jira issues'. Map technical scopes to human-readable descriptions. |
| Minimal scope | Request only what is needed for the current task. Use incremental consent — add scopes lazily, not upfront. |
| Revocation link | Consent screen must link to where user can revoke access. Atlassian: Connected Apps screen. Entra: My Apps (myapps.microsoft.com). |
| Duration disclosure | Tell user how long access will persist: 'This access will remain active until you revoke it or the agent task is complete.' |
| Data residency | For GDPR: disclose if token/data leaves EEA. Include in consent screen for non-EEA resource endpoints. |

### 13.3 Incremental Consent Pattern

Request scopes lazily — only when first needed — rather than all upfront. This reduces consent fatigue and is required when scopes span multiple products (e.g., Jira + Confluence + Graph in the same agent).

```
# Python: incremental consent check before each provider call async def ensure_atlassian_consent(user_oid: str, required_scopes: list[str], vault): """Check vault; if missing scope → trigger consent flow.""" cred = await vault.get(user_oid, provider="atlassian") if cred and all(s in cred.scopes for s in required_scopes): return # Consent already obtained for these scopes # Generate consent URL (scope union of existing + new) all_scopes = list(set((cred.scopes if cred else []) + required_scopes)) consent_url = ( "https://auth.atlassian.com/authorize" f"?client_id={ATLASSIAN_CLIENT_ID}" f"&scope;={' '.join(all_scopes + ['offline_access'])}" f"&redirect;_uri={AC_GATEWAY_CALLBACK}" f"&state;={generate_csrf_token(user_oid)}"\ # MUST be session-bound "&response;_type=code&prompt;=consent" ) # Surface consent URL to agent → agent surfaces to user # (In MCP: stream URL back via tool response; user clicks in chat UI) raise ConsentRequiredError(consent_url=consent_url, provider='atlassian')
```

## 14. OPERATIONAL RUNBOOK

### 14.1 Onboarding a New Agent Identity

| # | Step | Who | Command / Action |
|---|---|---|---|
| 1 | Register Blueprint | Identity Admin | az ad app create --display-name 'GenAI-AgentBlueprint-{name}' --sign-in-audience AzureADMyOrg Record Application (client) ID. |
| 2 | Create Blueprint Service Principal | Identity Admin | az ad sp create --id {blueprint_app_id} |
| 3 | Assign Managed Identity as FIC | Identity Admin | Graph API: POST /applications/{blueprint_id}/federatedIdentityCredentials { issuer: 'https://login.microsoftonline.com/{tid}/v2.0', subject: '{MSI_object_id}', audiences: ['api://AzureADTokenExchange'] } |
| 4 | Create Agent Identity (child) | Identity Admin | POST /servicePrincipals/{blueprint_sp_id}/agentIdentities { displayName: 'GenAI-AgentIdentity-{instance}' } — via MS Graph beta API |
| 5 | Grant delegated permissions | Identity Admin + Security approval | POST /oauth2PermissionGrants { clientId: {agent_identity_sp_id}, resourceId: {graph_sp_id}, scope: 'Mail.Read Calendars.Read' } |
| 6 | Set Conditional Access policy | Security | Entra portal → Security → Conditional Access → New policy → assign to Blueprint service principal |
| 7 | Register OAuth app in Atlassian | Builder | developer.atlassian.com → Create → OAuth 2.1 app → add scopes → set callback to AC Gateway URL |
| 8 | Register OAuth app in GHE | Builder | GHE Admin → Settings → Developer Settings → OAuth Apps → register with callback URL |
| 9 | Store app credentials in Key Vault | Platform/SRE | az keyvault secret set --vault-name {kv} --name 'atlassian-client-secret' --value {secret} Never put secrets in appsettings.json or environment variables. |
| 10 | Smoke test | Engineer | Run reference implementation test harness: verify T1 acquisition → OBO exchange → resource API call → audit log entry appears. |

### 14.2 Revoking User Access

| Scenario | Action | Where |
|---|---|---|
| User requests revoke | Set revoked_at in vault; delete refresh_token | Vault API; also revoke at provider (Atlassian Connected Apps, GHE OAuth token) |
| User leaves org | Entra offboarding disables user account → all delegated tokens invalidated automatically for Graph/Entra-backed resources | Entra offboarding + vault sweep job |
| Security incident | Revoke all credentials for agent_id: UPDATE agent_credentials SET revoked_at=NOW() WHERE agent_id='{id}' | Vault DB + alert SRE team |
| Agent decommissioned | Disable Agent Identity in Entra → revoke all app-level grants → delete vault entries for agent_id | Entra portal + vault DB + KV secret deletion |
| Atlassian revoke | User: Atlassian → Profile → Connected Apps → Revoke. Admin: can only uninstall app (not per-user revoke — Atlassian limitation) | Atlassian portal |

| Scenario | Action | Where |
|---|---|---|
| GHE revoke | User: GHE → Settings → Applications → Authorized OAuth Apps → Revoke. API: DELETE /applications/{client_id}/tokens/{token} | GHE portal or API |

## 14.3 Rotating the Blueprint FIC Credential (Certificate)

```
# Certificate rotation (zero-downtime, overlap window) # Step 1: Generate new cert (90-day validity) openssl req -x509 -newkey rsa:2048 -keyout new_key.pem -out new_cert.pem \ -days 90 -nodes -subj '/CN=GenAI-AgentBlueprint' # Step 2: Upload new cert to Entra App Registration (adds alongside old cert) az ad app credential reset --id {blueprint_app_id} --cert @new_cert.pem --append # Step 3: Update Key Vault with new cert+key (new version, old version still accessible) az keyvault certificate import --vault-name {kv} \ --name 'blueprint-cert' --file new_cert_bundle.pfx # Step 4: Deploy new version of agent service (picks up new KV cert version via MSI) # Step 5: Verify new cert works in staging → promote to prod # Step 6: Remove OLD cert from Entra App Registration after 48hr overlap az ad app credential delete --id {blueprint_app_id} --key-id {old_key_id} # Step 7: Delete old KV cert version # (Keep for 30 days for rollback before permanent deletion) az keyvault certificate delete --vault-name {kv} --name 'blueprint-cert' # Then purge after soft-delete retention period
```

## 14.4 Monitoring & Alerting

| Alert | Metric / Signal | Threshold | Action |
|---|---|---|---|
| Token refresh failure rate | vault.refresh.errors / vault.refresh.total | > 5% over 5min | Page SRE; check provider status; check lock contention |
| Refresh lag spike | P99 latency of ensure_fresh_token() | > 3s | Check distributed lock; scale refresh sweeper workers |
| 401 rate from downstream APIs | http.client.4xx{status=401} by provider | > 1% of calls | Token vault out of sync; trigger full re-consent for affected users |
| Vault decrypt audit anomaly | Key Vault: >N decrypts in 1min by non-agent identity | N defined by Security | Potential credential theft; freeze agent; rotate all DEKs |
| Consent dropoff | consent.started - consent.completed | > 30% dropoff | Investigate UX; scope too broad? Consent screen too alarming? |
| Revoked token still used | vault.access_revoked_credential_attempted | Any occurrence | Critical: agent not checking revoked_at flag; deploy fix immediately |
| Agent identity anomaly | Entra Identity Protection: risk score on agent SP | Medium or higher | Disable agent identity; investigate; re-provision after root cause |

# 15. INCIDENT RESPONSE PLAYBOOK

## 15.1 Incident Classification

| Severity | Definition | Examples | SLA |
|---|---|---|---|
| P0 — Critical | Agent acting outside authorised scope; data exfiltration suspected | Agent accessing resources user never consented to; token seen in external logs | Immediate: disable agent identity within 15min; notify CISO |
| P1 — High | Token leak or compromise confirmed | Refresh_token in application logs; vault database exposed | 1hr: rotate all tokens; notify affected users; Security investigation |
| P2 — Medium | Auth flow broken for subset of users or providers | Atlassian token refresh failing; GHE OAuth app rate-limited | 4hr: restore service; root cause within 24hr |
| P3 — Low | Individual user consent issue or single token expiry | User reports agent cannot access their Jira | 1 business day: diagnose and re-prompt consent if needed |

## 15.2 P0 Response Sequence — Token Compromise

```
## IMMEDIATE (< 15 minutes) 1. Disable Agent Identity in Entra (blocks all new token acquisitions): az ad sp update --id {agent_identity_sp_id} --set accountEnabled=false 2. Revoke all active credentials in vault: UPDATE agent_credentials SET revoked_at = NOW() WHERE agent_id = '{compromised_agent_id}' AND revoked_at IS NULL; 3. Invalidate refresh tokens at providers: # Atlassian: for each affected user POST https://auth.atlassian.com/oauth/revoke {token: {refresh_token}, client_id, client_secret} # Graph: revoke all refresh tokens for user (requires admin) POST https://graph.microsoft.com/v1.0/users/{user_id}/revokeSignInSessions # GHE: DELETE /applications/{client_id}/tokens/{access_token} ## SHORT-TERM (< 1 hour) 4. Rotate Blueprint certificate (Section 14.3) 5. Rotate all encrypted DEKs in Key Vault 6. Review Entra audit logs: filter by agent_identity_sp_id, last 72 hours 7. Export Key Vault access log: identify any non-authorised decrypt operations 8. Notify affected users; provide re-consent instructions ## INVESTIGATION (< 24 hours) 9. Trace token origin: which service first handled the compromised token? 10. Check: was token logged anywhere? (structured log search: grep for 'eyJ' pattern) 11. Root cause: credential in env var? Memory dump? Log injection? Prompt injection? 12. Post-incident review; update threat model; add detection rule for root cause vector
```

## 15.3 Common Failure Modes & Fixes

| Failure | Entra error code | Root cause | Fix |
|---|---|---|---|
| OBO exchange fails | AADSTS82001 | Using RFC 8693 token-exchange grant (wrong grant type) | Switch to jwt-bearer grant for OBO: grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer |
| Blueprint token fails | AADSTS700211 | Scope format wrong | Always use /.default in both exchange steps, not individual scope strings |
| Agent permission 403 | 403 Forbidden | Permission granted to Blueprint SP instead of Agent Identity SP | Grant permissions to Agent Identity SP; Admin consent propagation may take 30-120s — retry with backoff |
| Atlassian 401 on API call | 401 Unauthorized | access_token expired; refresh_token not rotated atomically | Verify vault rotate() is atomic; check optimistic locking on token_version |
| Atlassian refresh 400 invalid_grant | 400 invalid_grant | Refresh token already used (race condition) or rotated | Per-(user,provider) distributed lock prevents this; check lock expiry > token TTL |
| GHE 401 | 401 | No refresh token for GHE OAuth App; token expired | Switch to GitHub App installation tokens (1hr TTL, auto-renewable) or prompt re-auth |

| Failure | Entra error code | Root cause | Fix |
| --- | --- | --- | --- |
| Sidecar 403 from internal call | N/A | Sidecar exposed on LoadBalancer ingress; external caller | Sidecar must only bind to localhost:7000; block all external network access at pod/container level |

## 16. OWASP AGENTIC TOP 10 — AUTH CONTROLS MAPPING

The OWASP Agentic Top 10 defines the primary risk categories for autonomous AI agents. The following table maps each risk to the specific controls provided by the 3LO + AC Gateway pattern.

| OWASP Agentic | Risk | Control in this architecture |
|---|---|---|
| A01 — Excessive Agency | Agent performs actions beyond what user authorised | 3LO: tokens bounded by intersection of user permissions + agent-granted scopes. PDP policy check at gateway before every tool call. CIBA gate for high-risk actions. |
| A02 — Prompt Injection | Malicious input tricks agent into misusing its token | Token never in LLM context. Gateway (not agent) executes API calls. PDP evaluates action independent of LLM output — LLM is not trusted for auth decisions. |
| A03 — Insecure Output Handling | Token or credential leaks in agent response | Structured logging middleware strips all token-like strings (regex: Bearer\s+eyJ[A-Za-z0-9._-]+). Agent receives API results, not raw tokens. |
| A04 — Insecure Plugin Design | Tool/plugin has overly broad permissions | Per-scope consent: each tool declares required scopes. Incremental consent: scopes requested only when tool first used. Tool catalog has explicit risk tiers. |
| A05 — Improper Error Handling | Error messages leak credential details | Error handler strips token values from exceptions. Vault errors return generic 'auth_error' code; details in structured log only, not in agent response. |
| A06 — Insecure Credential Management | Credentials stored in code, env vars, or agent memory | AES-256-GCM encrypted vault. MSI/FIC — no secrets in config. Sidecar pattern: agent never handles raw tokens. Revoke on task completion. |
| A07 — Lack of Audit Logging | No record of what agent did or on whose behalf | Every gateway API call logs: agent_id, user_oid, scope, resource, method, response_code, timestamp. Key Vault logs every decrypt. Entra audit log for token issuance. |
| A08 — Insecure Defaults | New agent deployments start with broad access | Blueprint defines minimal scope set. New Agent Identities inherit Blueprint's Conditional Access. Access Packages require explicit approval for scope additions. |
| A09 — Outdated Dependencies | Agent SDKs with known vulnerabilities | Dependabot / Snyk on Microsoft.Identity.Web, MSAL packages. FIC + MSI means credential exposure reduced if SDK is compromised (no static secret to steal). |
| A10 — Insufficient Rate Limiting | Agent can call APIs at machine speed without throttle | AC Gateway enforces per-(agent_id, provider) rate limits. Token TTL constrains burst windows. CIBA imposes human-pace approval on high-frequency dangerous actions. |

## 17. CONFLUENCE SPECIFICATION TEMPLATE (Ready to Copy)

The following is a ready-to-use Confluence page template for the full EntraID 3LO Spec. Copy this structure into a new Confluence page under GENAI Agent Foundry → GENAITB-137.

```
H1: EntraID 3LO: AC Gateway + AC Identity — User-Delegated Auth for Agents

||Metadata||Value||
|Epic|GENAIAF-562|
|Status|DRAFT — pending AC Gateway whitelisting (GENAITB-201)|
|Owner|[Your name]|
|Security reviewer|[Security/Identity team contact]|
|Last updated|June 2026|

H2: 1. Problem Statement
_[Paste Section 1 content from Volume 1]_

H2: 2. Identity Architecture
_[Agent Identity Blueprint / Identity / User three-tier model; FIC+MSI chain]_

H2: 3. Token Flow Diagrams
H3: 3.1 3LO Consent Flow
{code:language=text}
User → [consent screen] → Auth Server → code → AC Gateway → token exchange → access_token +
refresh_token
{code}

H3: 3.2 OBO Token Chain
{code:language=text}
Client App → [user token Tc] → Agent → Blueprint acquires T1 (FIC/MSI)
→ Agent Identity OBO (T1+Tc) → resource token
{code}

H2: 4. AC Gateway Design
_[Three-plane architecture; credential vault schema; proactive refresh at 80% TTL]_

H2: 5. M365 Graph Integration
H3: Required scopes (minimum)
|| Scope || Access ||
| User.Read | Read profile |
| Mail.Read | Read email (request only when mail tool used) |
| Calendars.ReadWrite | Calendar access (request only when calendar tool used) |

H2: 6. GHE Integration
_[OAuth App vs GitHub App decision; callback URL whitelist; token TTL handling]_

H2: 7. Atlassian Integration
_[Scope table; cloudId discovery; offline_access requirement; rotating refresh token handling]_

H2: 8. Consent Flow UX
_[Admin pre-consent vs user 3LO vs CIBA; consent screen requirements; incremental consent]_

H2: 9. Security Controls
_[PKCE/CSRF; threat model; OWASP Agentic Top 10 mapping; token isolation]_

H2: 10. Reference Implementation
H3: 10.1 .NET (Microsoft.Identity.Web)
{code:language=csharp}
// [paste Section 11.2 code]
{code}

H3: 10.2 Python (MSAL direct)
{code:language=python}
# [paste Section 11.3 code]
{code}
```

**H3:** 10.3 Python (Sidecar pattern)
{code:language=python}
# [paste Section 11.4 code]
{code}

**H2:** 11. Operational Runbook
_[Section 14: Onboarding, revocation, cert rotation, monitoring alerts]_

**H2:** 12. Incident Response
_[Section 15: P0-P3 classification; P0 sequence; common failure codes]_

**H2:** 13. Dependency Tracker
|| Dependency || Status || Ticket || Owner ||
| AC Gateway whitelisting | Blocked | GENAITB-201 | Platform/Infra |
| AC Identity whitelisting | Blocked | GENAITB-201 | Platform/Infra |
| FT/Entra self-service | Manual (pending process) | — | Security/Identity |

**H2:** 14. Open Questions
_[Section 10 open questions: OQ-1 through OQ-6]_

**H2:** 15. Glossary
|| Term || Definition ||
| 3LO | Three-Legged OAuth — Auth Code Grant with user consent |
| 2LO | Two-Legged OAuth — Client Credentials (no user) |
| OBO | On-Behalf-Of — RFC 8693 token exchange for delegation |
| FIC | Federated Identity Credential — secretless auth for Blueprint |
| MSI | Managed Service Identity — Azure-managed credential |
| PKCE | Proof Key for Code Exchange — prevents code injection |
| CIBA | Client-Initiated Backchannel Auth — async user approval |
| MCP | Model Context Protocol — standard for agent tool connections |
| Blueprint | Agent Identity Blueprint — parent app registration template |
| DEK/KEK | Data/Key Encryption Key — vault encryption hierarchy |

## 18. QUICK REFERENCE — ONE-PAGE CHEAT SHEET

### Token Chain at a Glance

```
INTERACTIVE AGENT (3LO OBO): Client App (user signs in) → sends user token Tc to Agent Blueprint
POST /token grant=client_credentials FIC/MSI → T1 AgentIdentity POST /token grant=jwt-bearer
assertion=Tc client_assertion=T1 → RESOURCE TOKEN Agent calls downstream API with RESOURCE TOKEN
AUTONOMOUS AGENT (2LO): Blueprint POST /token grant=client_credentials FIC/MSI → T1 AgentIdentity
POST /token grant=client_credentials fmi_path=AgentIdentity client_assertion=T1 → APP TOKEN Agent
calls downstream API with APP TOKEN SIDECAR SHORTCUT: Agent → GET
http://localhost:7000/AuthorizationHeader/Graph ?AgentIdentity={guid}&AgentUserId;={oid}
Authorization: Bearer {user_token} Sidecar performs the full chain; returns 'Bearer eyJ...'
```

### Do / Don't Reference

| ✓ DO | ✗ DON'T |
|---|---|
| Use FIC + MSI for Blueprint credential | Use client_secret in production |
| Use jwt-bearer grant for OBO exchange | Use RFC 8693 token-exchange grant (AADSTS82001) |
| Use /.default scope in all exchange steps | Use individual scope strings in exchange |
| Store refresh_token encrypted in vault | Store token in env var, config file, or LLM prompt |
| Proactively refresh at 80% TTL with lock | Wait for 401 to trigger refresh (causes cascade) |
| Rotate refresh_token atomically (new+invalidate old) | Reuse old refresh_token after rotation (invalid_grant) |
| Grant permissions to Agent Identity SP | Grant permissions to Blueprint SP (wrong level) |
| Add offline_access scope for background agents | Omit offline_access (no refresh_token issued) |
| Revoke refresh_token on task completion | Let agent retain tokens between unrelated tasks |
| Log: agent_id + user_oid + scope + resource + ts | Log raw token values (even truncated) |
| Use CIBA for irreversible/high-risk actions | Let agent perform bulk/destructive actions autonomously |
| Use incremental consent (scope on demand) | Request all scopes upfront regardless of task |

### Provider-Specific Gotchas

| Provider | Critical gotcha |
|---|---|
| Atlassian | MUST include offline_access in scope to get refresh_token. Token rotation: new refresh_token issued on every refresh — always store the latest or you get 400 invalid_grant. |
| GHE OAuth App | No refresh_token — access tokens are long-lived by default. Prefer GitHub App installation tokens (1hr, auto-renewable, fine-grained). |
| MS Graph (delegated) | OBO requires audience of Tc to match Blueprint client_id. token_version/nonce mismatch causes AADSTS70011. |
| Entra Agent ID | Never use Azure CLI tokens for Agent Identity API calls (Directory.AccessAsUser.All → hard 403). Permission grant propagation: retry 403s with 30-120s backoff after admin consent. |
| All providers | Per-(user,provider) distributed lock on refresh is not optional at scale. Race condition without it corrupts vault state and cascades to service outage. |